Sign in

pytorch / pytorch Public

Notifications Fork 23.6k Star 88.1k

Code Issues 5k+

[JIT] jit can not recognize the imported function #38964

New issue

Open



chenbohua3 opened on May 24, 2020 · edited by pytorch-probot

Edits

Bug

JIT can not recognize the imported function when I want to script a class outside the file where it is defined.

To Reproduce

Suppose we have the following file structure:

```
main.py
files/
    __init__.py
    a.py
    b.py
```

and each file contains:

```
# a.py
from .b import f

class MyObj(object):
    def __init__(self):
        pass

    def __call__(self, x):
        return f(x)

# b.py
def f(x):
    return x

# __init__.py
from .a import *
from .b import *
```

Assignees

qihqi

Labels

oncall: jit triaged

Type

No type

Projects

No projects

Milestone

No milestone

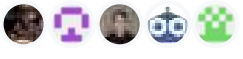
Relationships

None yet

Development

No branches or pull requests

Participants



+1

```
# main.py:
import torch
from files import MyObj

torch.jit.script(MyObj)
```

then we will get:

```
RuntimeError:
undefined value f:
  File "a.py", line 8
    def __call__(self, x):
        return f(x)
               ~ <--- HERE
```

function `f` must be imported explicitly in `main.py` to let jit know what is `f`

```
from files import f
```

Expected behavior

There is no need to import function `f` explicitly

Environment

- PyTorch Version (e.g., 1.0): master
- OS (e.g., Linux): Linux
- How you installed PyTorch (`conda`, `pip`, source): source
- Build command you used (if compiling from source): as described in here
- Python version: 3.7.6
- CUDA/cuDNN version: 10.0 / 7.6.5

cc @suo

facebook-github-bot added oncall: jit on May 24, 2020

jamesr66a added triaged on May 26, 2020



jamesr66a on May 26, 2020 Collaborator

Hi @chenbohua3!

Thanks for the report! This indeed seems like a bug and is an

Open **[JIT] jit can not recognize the imported function** #38964

1

ppwwyyxx mentioned this on Jul 15, 2020

Make RPN scriptable.
facebookresearch/detectron2#1765

gmagogsfm on Apr 2, 2021 Contributor

Tested at close to HEAD, this is still reproducible.

ansley assigned Lilyjjo on Apr 6, 2021

gmagogsfm assigned gmagogsfm and unassigned Lilyjjo on Aug 11, 2021

gmagogsfm on Aug 11, 2021 Contributor

Claiming for a bootcamp task

qihqi on Aug 11, 2021 Contributor

The cause seems to be we are using name resolution callback created by `_jit_internal.createResolutionCallbackFromFrame` (https://github.com/pytorch/pytorch/blob/master/torch/jit/_script.py#L1266). This works by getting python call frame in the level of when `torch.jit.script` is called; in this case, is the `main.py`. `main.py` does not define symbol `f` and therefore it fails. Using createResolutionCallbackFromFrame is only correct if `torch.jit.script` is applied as decorator on the class.

This also explains why import `f` solves the issue. Note that this behavior is dangerous because, if main happens to define a different `f` (via. another `def f()` or even `f = ...`) then the current name resolution would happily use the wrong one.

Note that for functions does not have the issue because name resolution callback for functions uses `createResolutionCallbackFromClosure` and uses full closure of the function.

Potential solutions:

1. Use createResolutionCallbackFromClosure with any method of the class.

2. Use the module which the class is defined (by calling `__import__(obj.__module__)`) and use that for name resolution.

Note that 1. would not work if a class has no method at all, example:

```
class A:
  pass
```

that case shall be dealt separately by using any (for example, the current) resolution callback. Because a class with no behavior doesn't depend on any names by definition.

and 2. wouldn't work if the class is further contained in some closure.

```
def make_class():
  def xyz():
     ...
  class Inner:
     ...# uses xyz in some method
  return Inner
```

Overall seems like 1. is the most correct one. I will implement it shortly.



gmagogsfm on Aug 12, 2021 Contributor •••

> The cause seems to be we are using name resolution callback created by `_jit_internal.createResolutionCallbackFromFrame` (https://github.com/pytorch/pytorch/blob/master/torch/jit/_script.py#L1266). This works by getting python call frame in the level of when `torch.jit.script` is called; in this case, is the `main.py`. `main.py` does not define symbol `f` and therefore it fails. Using createResolutionCallbackFromFrame is only correct if `torch.jit.script` is applied as decorator on the class.
>
> This also explains why import `f` solves the issue. Note that this behavior is dangerous because, if main happens to define a different `f` (via. another `def f()` or even `f = ...`) then the current name resolution would happily use the wrong one.
>
> Note that for functions does not have the issue because name resolution callback for functions uses `createResolutionCallbackFromClosure` and uses full closure of the function.

> Potential solutions:
>
> 1. Use createResolutionCallbackFromClosure with any method of the class.
> 2. Use the module which the class is defined (by calling `__import__(obj.__module__)`) and use that for name resolution.
>
> Note that 1. would not work if a class has no method at all, example:
>
> ```
> class A:
>   pass
> ```
>
> that case shall be dealt separately by using any (for example, the current) resolution callback. Because a class with no behavior doesn't depend on any names by definition.
>
> and 2. wouldn't work if the class is further contained in some closure.
>
> ```
> def make_class():
>   def xyz():
>      ...
>   class Inner:
>      ...# uses xyz in some method
>   return Inner
> ```
>
> Overall seems like 1. is the most correct one. I will implement it shortly.

Nice analysis.

Agreed #1 seems like the way to go. A class without any methods is an extreme case that shouldn't matter because if it doesn't have any methods, it wouldn't need to reference a symbol name.

gmagogsfm assigned qihqi and unassigned gmagogsfm on Aug 12, 2021

qihqi mentioned this on Aug 13, 2021

Fix jit can not recognize the imported function #63248

qihqi added a commit that references this issue on Aug 13, 2021

`Fix jit can not recognize the imported fu …`

`8fdaa07`

12 remaining items

Load more

qihqi added 11 commits that reference this issue on Sep 30, 2021

`Update on "Fix jit can not recognize the ...` `e3e0e79`

`Update base for Update on "Fix jit can nc ...` `fe54709`

`Update on "Fix jit can not recognize the ...` `f49e1ad`

`Fix jit can not recognize the imported fu ...` `a5e2cc7`

`Fix jit can not recognize the imported fu ...` `786ed45`

`Update base for Update on "Fix jit can nc ...` `68312da`

`Update on "Fix jit can not recognize the ...` `a1bb149`

`Fix jit can not recognize the imported fu ...` `310e7ec`

`Update base for Update on "Fix jit can nc ...` `8f0a30f`

`Update on "Fix jit can not recognize the ...` `6d0135c`

qihqi added 3 commits that reference this issue on Dec 21, 2021

`Fix jit can not recognize the imported fu ...` `a7ca278`

`Update base for Update on "Fix jit can nc ...` `bf8f1a2`

`Update on "Fix jit can not recognize the ...` `589c504`

pmeier mentioned this on Feb 17, 2022

try api call logging with decorator
pytorch/vision#5424

Sign up for free **to join this conversation on GitHub.** Already have an account? Sign in to comment